Assignment 5: Perceptron

Gaurav Patil

17/04/2026

1 Introduction

This assignment studies binary classification using a perceptron on a two-feature dataset from `data.csv`. The work is divided into two parts. In Part 1, the perceptron is trained using the heuristic update rule specified in the assignment. In Part 2, the perceptron is trained using the misclassification-based update rule labeled as the gradient descent approach in the handout. For both parts, the dataset is plotted, the evolution of the decision boundary is visualized, and hyperparameters are varied to study their effect on performance.

Beyond the minimum requirements, this report includes both learning-rate and epoch experiments for both parts, quantitative summary tables, and a direct comparison between the two training approaches.

2 Dataset

The dataset contains 100 samples, 2 numerical input features, and 1 binary label column. The labels are `0` and `1`, and there are no missing values. Because there are exactly two features, the data can be visualized directly in a 2D scatter plot and the perceptron decision boundary can be drawn as a line.

The scatter plot shows that Class 0 is concentrated more toward the lower-left region, while Class 1 is mostly clustered in the upper-right region. This suggests that a linear decision boundary should perform reasonably well, although perfect separation may not be possible due to the overlap.

3 Perceptron Model

For both parts, the perceptron uses a linear combination of the inputs followed by a sigmoid function:

$$z = w^T x + b \tag{1}$$

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} \tag{2}$$

where w is the weight vector, b is the bias term, and $\hat{y}$ is the model output.

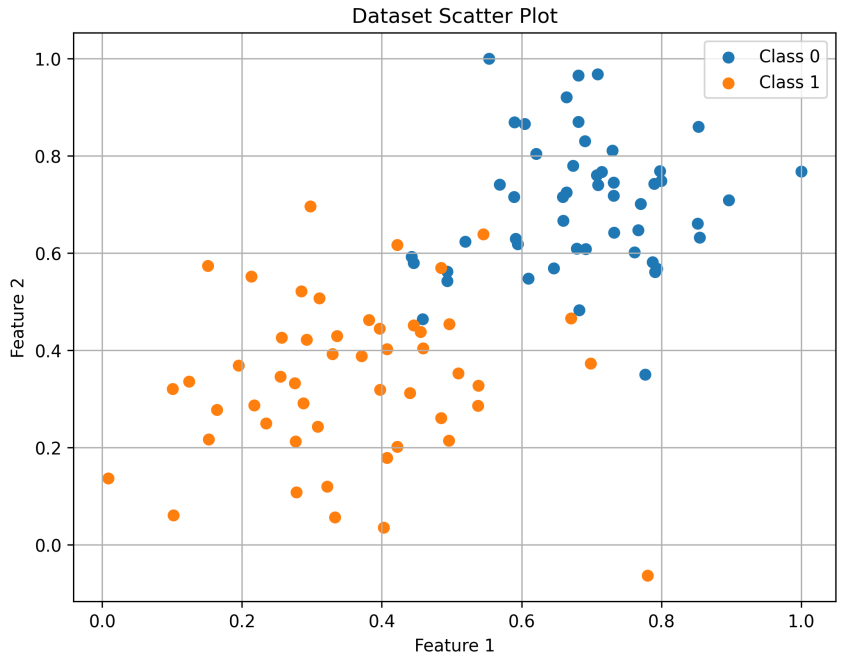


Figure 1: Scatter plot of the dataset from `data.csv`. The two classes are visually separable, although there is some overlap near the center of the feature space.

The decision boundary is given by:

$$w_1 x_1 + w_2 x_2 + b = 0 \tag{3}$$

which can be rearranged as:

$$x_2 = -\frac{w_1 x_1 + b}{w_2} \tag{4}$$

provided that $w_2 \neq 0$.

In all decision-boundary plots, the assignment's required color convention is used:

- initial boundary: red
- intermediate boundaries: dashed green
- final boundary: black

4 Implementation Summary

The notebook was written in Google Colab using Python, NumPy, pandas, and Matplotlib. The workflow was organized into the following stages:

1. load and inspect the dataset,
2. create a scatter plot of the raw data,

3. implement common helper functions such as `sigmoid`, prediction functions, log loss, and decision-boundary plotting,
4. implement Part 1 training,
5. implement Part 2 training,
6. run controlled experiments for learning rate and epochs,
7. save all figures for later use in the report.

Selected code snippets are shown below.

4.1 Core helper functions

Listing 1: Sigmoid, prediction, and log loss helpers.

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def predict_probability(X, w, b):
    z = np.dot(X, w) + b
    return sigmoid(z)

def predict_class(X, w, b, threshold=0.5):
    probs = predict_probability(X, w, b)
    return (probs >= threshold).astype(int)

def compute_log_loss(y_true, y_prob, eps=1e-12):
    y_prob = np.clip(y_prob, eps, 1 - eps)
    return -np.mean(y_true * np.log(y_prob) + (1 - y_true) * np.log(1 - y_prob))
```

4.2 Part 1 training rule

Listing 2: Heuristic update rule used in Part 1.

```
def train_part1_heuristic(X, y, learning_rate=0.1, epochs=50, seed=42):
    np.random.seed(seed)
    n_features = X.shape[1]
    w = np.random.randn(n_features)
    b = np.random.randn()

    history = [(w.copy(), b)]
    epoch_losses = []

    for epoch in range(epochs):
        for i in range(len(X)):
            x_i = X[i]
```

```
            y_i = y[i]

            y_hat = sigmoid(np.dot(w, x_i) + b)
            error = y_i - y_hat

            b = b + learning_rate * error
            w = w + learning_rate * error * x_i

        y_prob = predict_probability(X, w, b)
        loss = compute_log_loss(y, y_prob)
        epoch_losses.append(loss)
        history.append((w.copy(), b))

    return w, b, history, epoch_losses
```

4.3 Part 2 training rule

Listing 3: Misclassification-based update rule used in Part 2.

```
def train_part2_gradient_descent(X, y, learning_rate=0.1, epochs=50, seed=42):
    np.random.seed(seed)
    n_features = X.shape[1]
    w = np.random.randn(n_features)
    b = np.random.randn()

    history = [(w.copy(), b)]
    epoch_losses = []
    epoch_accuracies = []
    misclassified_counts = []

    for epoch in range(epochs):
        misclassified = 0
        for i in range(len(X)):
            x_i = X[i]
            y_i = y[i]
            y_pred = int(sigmoid(np.dot(w, x_i) + b) >= 0.5)

            if y_pred != y_i:
                misclassified += 1
                if y_pred == 0:
                    b = b + learning_rate
                    w = w + learning_rate * x_i
                else:
                    b = b - learning_rate
                    w = w - learning_rate * x_i

        y_prob = predict_probability(X, w, b)
```

```
        y_class = (y_prob >= 0.5).astype(int)
        loss = compute_log_loss(y, y_prob)
        acc = np.mean(y_class == y)

        epoch_losses.append(loss)
        epoch_accuracies.append(acc)
        misclassified_counts.append(misclassified)
        history.append((w.copy(), b))

    return w, b, history, epoch_losses, epoch_accuracies, misclassified_counts
```

5 Part 1: Heuristic Approach

Part 1 follows the assignment's heuristic rule:

$$\hat{y} = \sigma(w^T x + b) \tag{5}$$

$$\text{error} = y - \hat{y} \tag{6}$$

$$b \leftarrow b + r(y - \hat{y}) \tag{7}$$

$$w_i \leftarrow w_i + r(y - \hat{y})x_i \tag{8}$$

where r is the learning rate.

5.1 Baseline result

The baseline configuration for Part 1 used learning rate $r = 0.1$ and 50 epochs. The final performance from this run was:

- final log loss: 0.193322
- accuracy: 0.93

5.2 Learning-rate experiments

To study the effect of learning rate, the number of epochs was fixed at 50 and three values were tested: 0.01, 0.1, and 1.0.

Table 1: Part 1 learning-rate experiments at 50 epochs.

Experiment	Final Log Loss	Accuracy
lr = 0.01, epochs = 50	0.414441	0.93
lr = 0.1, epochs = 50	0.193322	0.93
lr = 1.0, epochs = 50	0.277245	0.91

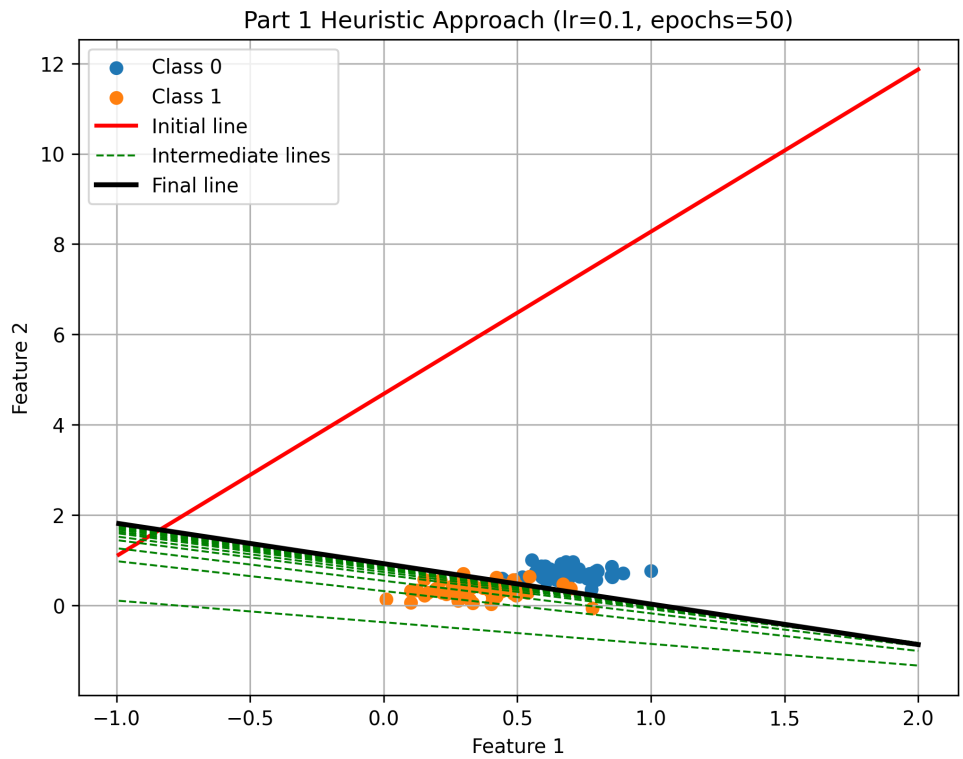
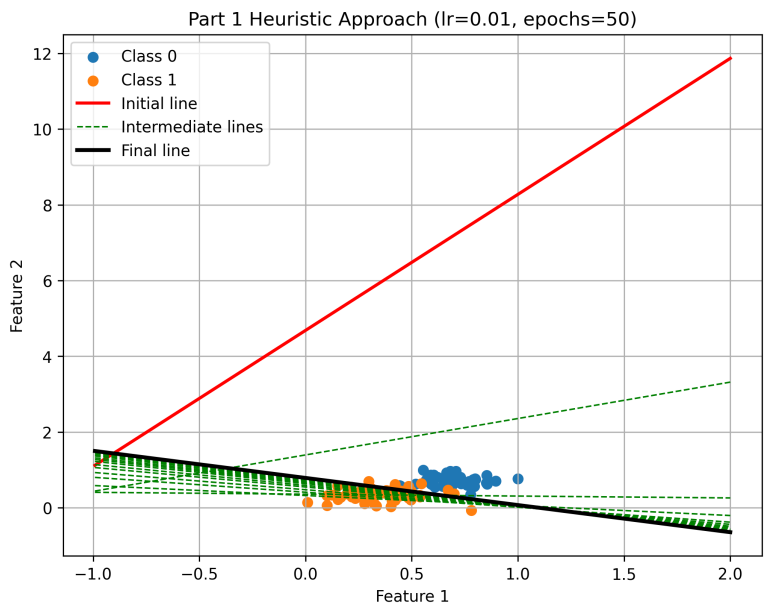
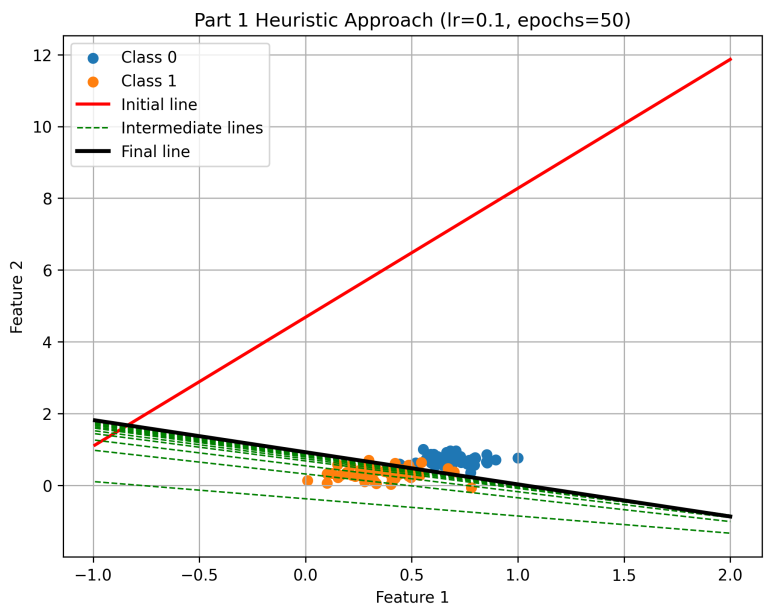


Figure 2: Part 1 baseline decision boundary evolution for learning rate 0.1 and 50 epochs. The final boundary is visually stable and separates the two clusters well.

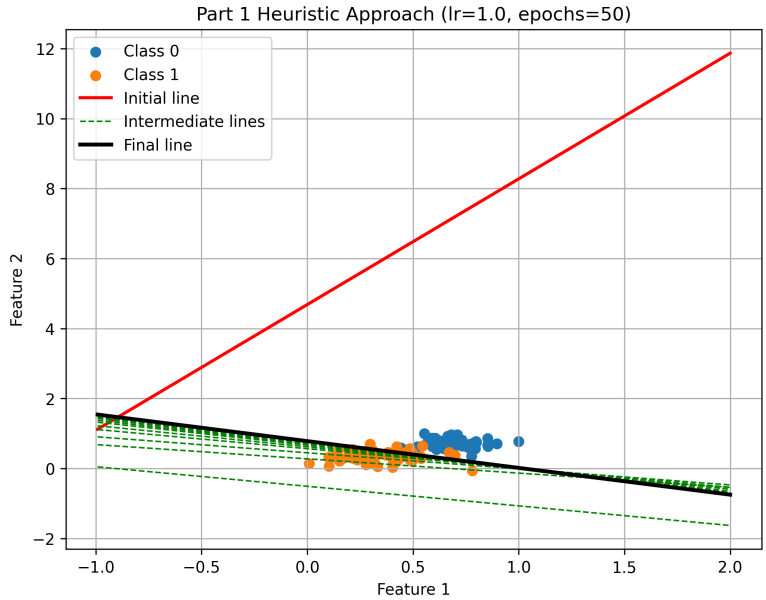


(a) lr = 0.01



(b) lr = 0.1

Figure 3: Part 1 decision-boundary evolution for different learning rates (first two runs).



(c) lr = 1.0

Figure 4: Part 1 decision-boundary evolution for different learning rates (final run).

These results show that the smallest learning rate, 0.01, learns more slowly and ends with noticeably higher loss. The medium learning rate, 0.1, gives the best overall balance, achieving the lowest loss among the three while maintaining the highest accuracy. The largest learning rate, 1.0, still learns a separating boundary, but its updates are more aggressive and the final accuracy drops slightly to 0.91.

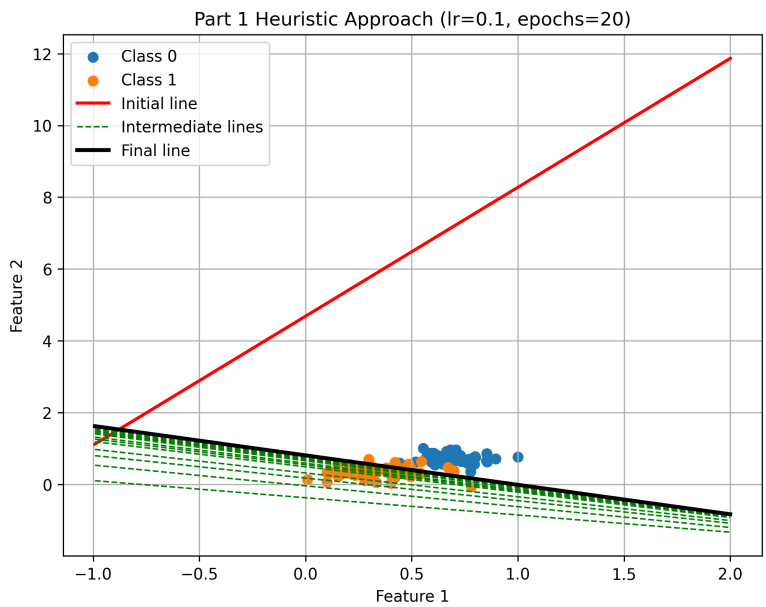
5.3 Epoch experiments

To study the effect of training duration, the learning rate was fixed at 0.1 and the model was trained for 20, 50, and 100 epochs.

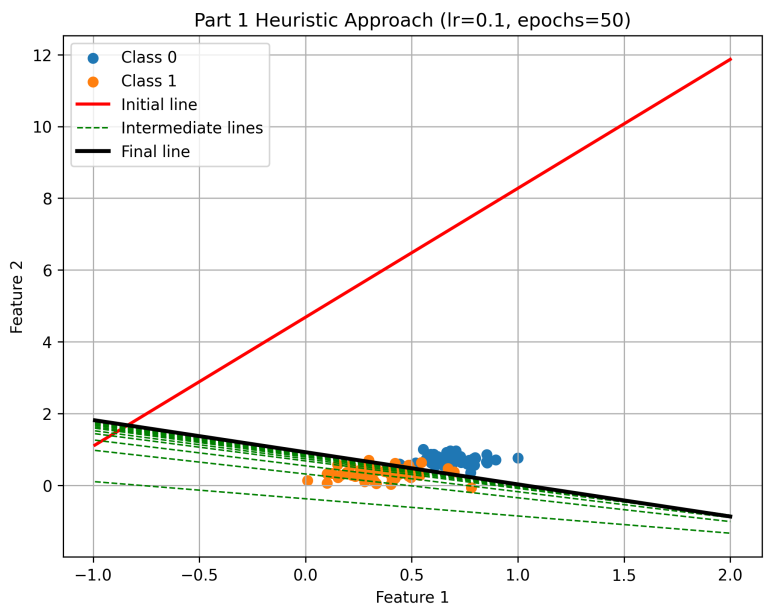
Table 2: Part 1 epoch experiments at learning rate 0.1.

Experiment	Final Log Loss	Accuracy
lr = 0.1, epochs = 20	0.282453	0.91
lr = 0.1, epochs = 50	0.193322	0.93
lr = 0.1, epochs = 100	0.161638	0.93

Increasing the number of epochs improves the final loss from 0.282453 at 20 epochs to 0.161638 at 100 epochs. Accuracy improves from 0.91 to 0.93 and then stabilizes. This suggests that additional passes through the data allow the heuristic update rule to refine the boundary further, although the marginal improvement in accuracy becomes small after 50 epochs.

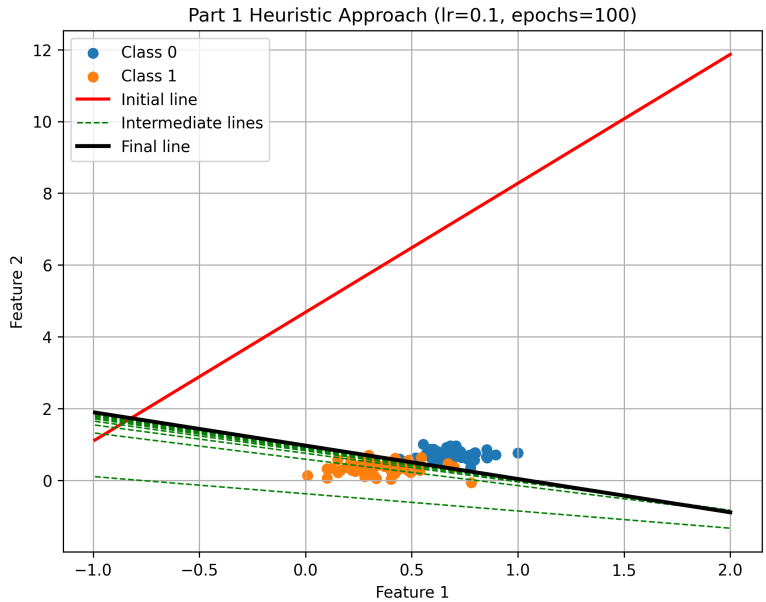


(a) 20 epochs



(b) 50 epochs

Figure 5: Part 1 decision-boundary evolution for different epoch counts (first two runs).



(c) 100 epochs

Figure 6: Part 1 decision-boundary evolution for different epoch counts (final run).

6 Part 2: Gradient Descent Approach from the Assignment

Part 2 follows the right-box rule from the handout. For each training point, the model first computes a class label. If the point is misclassified, the weights and bias are adjusted according to the predicted class:

$$\text{if } \hat{y}_{class} = 0 : b \leftarrow b + r, \quad w_i \leftarrow w_i + rx_i \tag{9}$$

$$\text{if } \hat{y}_{class} = 1 : b \leftarrow b - r, \quad w_i \leftarrow w_i - rx_i \tag{10}$$

In addition to boundary plots, Part 2 requires computation of log loss and an error plot every 10 epochs.

6.1 Baseline result

The baseline configuration for Part 2 also used learning rate $r = 0.1$ and 50 epochs. The final performance from this run was:

- final log loss: 0.595328
- accuracy: 0.92
- last-epoch misclassified count: 4

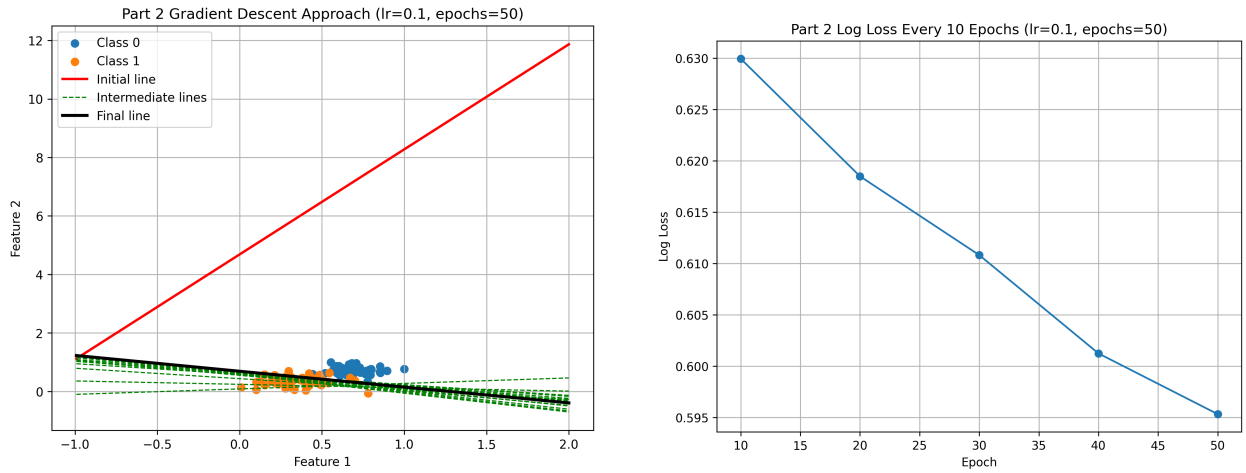


Figure 7: Part 2 baseline results for learning rate 0.1 and 50 epochs. Left: boundary evolution. Right: log loss every 10 epochs.

6.2 Learning-rate experiments

The number of epochs was fixed at 50 and the learning rates 0.01, 0.1, and 1.0 were tested.

Table 3: Part 2 learning-rate experiments at 50 epochs.

Experiment	Final Log Loss	Accuracy	Misclassified (last epoch)
lr = 0.01, epochs = 50	0.653952	0.92	8
lr = 0.1, epochs = 50	0.595328	0.92	4
lr = 1.0, epochs = 50	0.361025	0.79	4

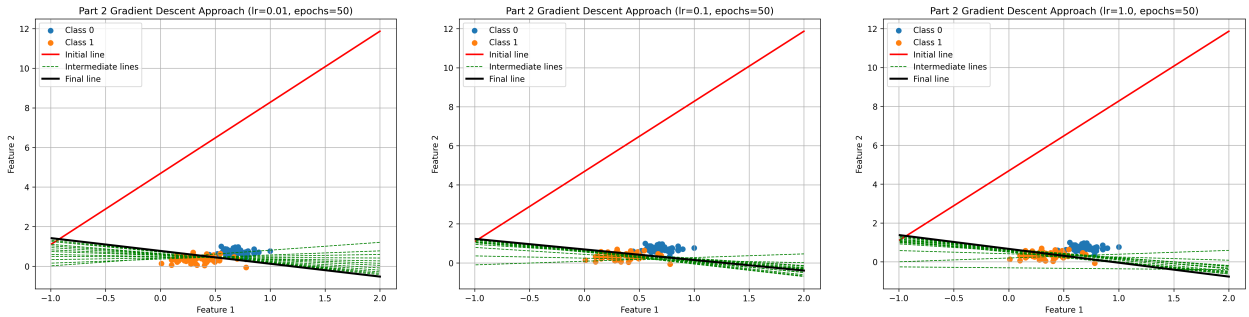
An interesting pattern appears in Part 2. The learning rate 1.0 gives the lowest log loss among these three runs, but its classification accuracy drops to 0.79. In contrast, both 0.01 and 0.1 achieve 0.92 accuracy. This indicates that a lower loss does not automatically guarantee the best class predictions under a fixed 0.5 threshold. For this dataset, 0.1 appears to be the best compromise because it improves loss relative to 0.01 while preserving strong accuracy.

6.3 Epoch experiments

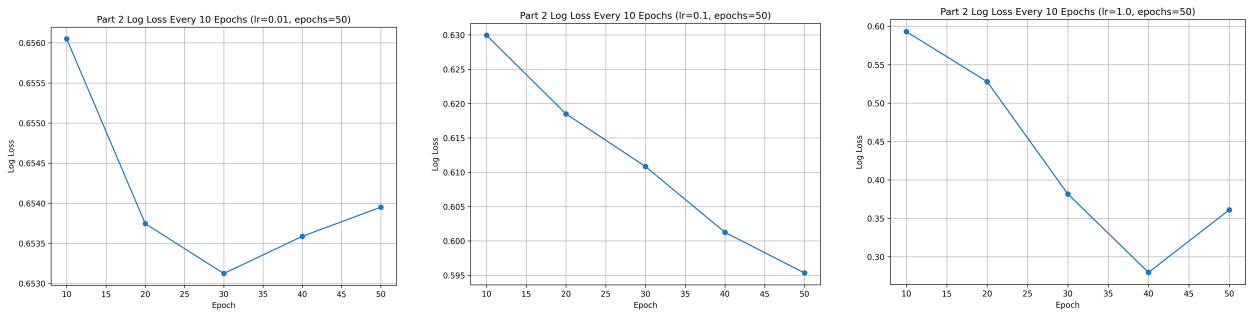
The learning rate was fixed at 0.1 and the model was trained for 20, 50, and 100 epochs.

Table 4: Part 2 epoch experiments at learning rate 0.1.

Experiment	Final Log Loss	Accuracy	Misclassified (last epoch)
lr = 0.1, epochs = 20	0.618487	0.77	4
lr = 0.1, epochs = 50	0.595328	0.92	4
lr = 0.1, epochs = 100	0.570548	0.81	6



Top row: boundary evolution for lr = 0.01, 0.1, and 1.0.



Bottom row: log-loss curves every 10 epochs for lr = 0.01, 0.1, and 1.0.

Figure 8: Part 2 learning-rate experiments: boundary evolution and loss curves.

For Part 2, increasing epochs does not produce a simple monotonic improvement in classification accuracy. Accuracy rises sharply from 0.77 at 20 epochs to 0.92 at 50 epochs, but then drops to 0.81 at 100 epochs. Meanwhile, log loss continues to decrease slightly. This again shows that optimizing a continuous probability-based loss and maximizing thresholded classification accuracy are related but not identical goals.

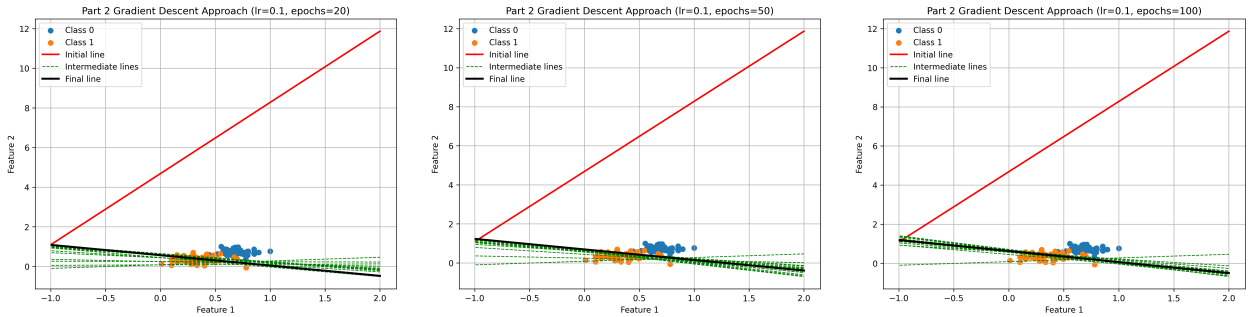
7 Direct Comparison of Part 1 and Part 2

The baseline configurations for both parts used learning rate 0.1 and 50 epochs, making them directly comparable.

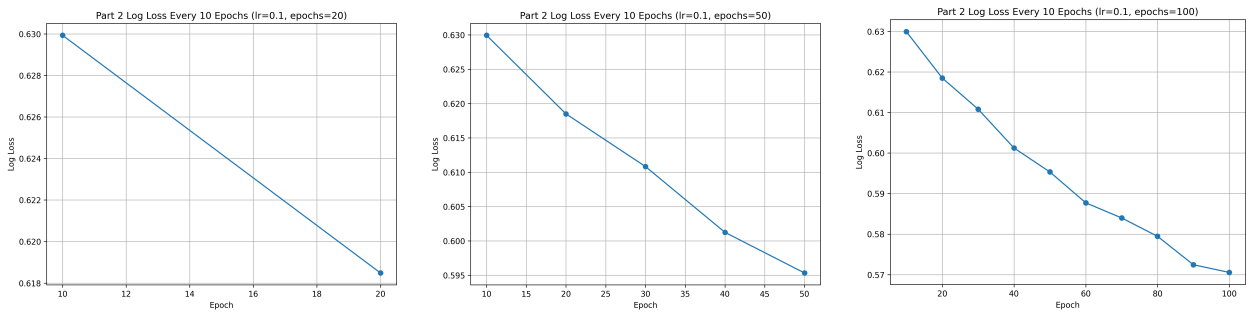
Table 5: Baseline comparison between Part 1 and Part 2.

Method	Learning Rate	Epochs	Final Log Loss	Accuracy
Part 1 Heuristic	0.1	50	0.193322	0.93
Part 2 Gradient Descent	0.1	50	0.595328	0.92

In this assignment, the Part 1 heuristic approach produced the better overall baseline result. It achieved lower loss and slightly higher accuracy than Part 2 under the same learning rate and number of epochs. It also appeared more stable under epoch scaling, since



Top row: boundary evolution for 20, 50, and 100 epochs.



Bottom row: log-loss curves every 10 epochs for 20, 50, and 100 epochs.

Figure 9: Part 2 epoch experiments: boundary evolution and loss curves.

its accuracy remained consistent from 50 to 100 epochs while Part 2 became less accurate at 100 epochs.

This does not mean Part 2 is ineffective. Rather, it suggests that on this specific dataset and with the chosen thresholding scheme, the heuristic update rule aligns better with the observed classification objective. Part 2 still provides useful insight because it demonstrates how a perceptron can be updated based only on classification mistakes, and it highlights the difference between decreasing log loss and improving final class predictions.

8 Discussion

Several conclusions can be drawn from the experiments:

1. The dataset is close to linearly separable, so both approaches are able to learn reasonable boundaries.

2. In both parts, a very small learning rate tends to learn more slowly.

3. In both parts, a learning rate of 0.1 gives the most balanced results overall.

4. A larger learning rate can make updates more aggressive. In Part 1 this slightly hurt accuracy, and in Part 2 it significantly reduced accuracy even though log loss decreased.

5. More epochs are not always better, especially for the Part 2 rule. The 100-epoch result shows that longer training can continue decreasing loss while making the thresholded classification result less accurate.

A useful interpretation is that the decision boundary can continue moving in ways that improve confidence values, but those movements may not improve how many points are correctly classified after thresholding at 0.5. This distinction is one of the most important lessons from the experiments.

9 Conclusion

This assignment implemented and analyzed two perceptron training strategies on the same binary classification dataset. Both methods were able to find meaningful separating boundaries, and both showed clear sensitivity to learning rate and number of epochs.

Part 1, the heuristic approach, delivered the strongest overall performance in these experiments. Its best tested configuration was learning rate 0.1 with 100 epochs, which achieved a final log loss of 0.161638 and accuracy of 0.93. Part 2, the assignment's gradient descent approach, produced its strongest balanced result at learning rate 0.1 and 50 epochs, achieving a final log loss of 0.595328 and accuracy of 0.92.

Overall, the experiments demonstrate that perceptron learning is highly dependent on hyperparameter choice, that visualizing decision-boundary evolution is useful for understanding training behavior, and that lower continuous loss does not always translate into higher classification accuracy.